Q1) You are building a smart irrigation system that must monitor soil moisture, control a water pump, and send data to a mobile application every 5 seconds. You have both an Arduino UNO and an ESP32. Which controller would you choose and why? Would your answer change if internet connectivity was not required?

**Controller Selection: ESP32**

I would choose the ESP32 because it has built-in Wi-Fi/Bluetooth and enough processing, timers, and ADC channels to read soil sensors every 5 seconds and send data to a mobile app; if internet connectivity is not required I would still prefer the ESP32 for its extra I/O and performance unless the project constraints (cost, ultra-low power, or very simple required features) favor the UNO.Here is a detailed breakdown of why:

- **Native Wireless Connectivity:** The primary requirement is to send data to a mobile application every 5 seconds. The ESP32 features **built-in Wi-Fi and Bluetooth (BLE)**, allowing it to connect directly to the internet (via Wi-Fi to a cloud broker like MQTT or Firebase) or directly to a smartphone (via BLE or Wi-Fi Access Point mode). The Arduino UNO has no native wireless capabilities and would require expensive, bulky external shields (like an ESP8266 or HC-05 module).
- **Performance and Memory:** Sending data every 5 seconds while simultaneously reading sensors and controlling an actuator requires handling network protocol stacks (like TCP/IP, MQTT, or HTTP). The ESP32 features a dual-core processor running at 160/240 MHz with 520 KB of SRAM. In contrast, the Arduino UNO runs at a mere 16 MHz with only 2 KB of SRAM, which can easily crash or run out of memory when handling modern network encryption and data transmission.
- **Power Management:** Irrigation systems are often deployed outdoors where battery or solar power is necessary. The ESP32 supports advanced **Deep Sleep modes** that turn off the network modules and cores when not in use, waking up instantly every 5 seconds to take a reading, send data, and sleep again. The Arduino UNO's power-saving modes are significantly less efficient.

**Would the answer change if internet connectivity was not required?**

**No, the core choice would still remain the ESP32, with a minor caveat.**

Here is how the scenario changes based on the remaining requirements:

1. **If a mobile application is still required:** Even without *internet* connectivity, the system must still communicate with a mobile app. The ESP32 can use **Bluetooth Low Energy (BLE)** or act as a **Wi-Fi Access Point (SoftAP)** to stream data directly to a local smartphone without needing an internet router. The Arduino UNO would still need external hardware to achieve this.

2. **If the mobile app requirement is also dropped (Pure Standalone System):** Only if the system becomes a completely offline, standalone automated loop (read moisture )toggle pump locally with no data transmission) would the **Arduino UNO** become a viable alternative. In that specific case, the UNO is simpler, highly reliable, operates at a standard 5V (which matches many standard relay boards natively), and avoids the unnecessary complexity of the ESP32.

Q2) A smart factory contains 500 sensors distributed across a 2 km area. Compare Wi-Fi, Bluetooth, LoRa, and MQTT-based communication. Which protocol or combination of protocols would you choose and why?

For a smart factory with 500 sensors spread across 2 km, I would choose **LoRa for the sensor layer and MQTT for the messaging layer**, rather than relying on Wi-Fi or Bluetooth alone. LoRa is the best fit for the long range and low power needs, while MQTT is ideal for moving sensor data from the gateway to software systems in a lightweight way.

# Comparison

| Technology | Best range | Power use | Data speed | Fit for 500 sensors over 2 km |
|---|---|---|---|---|
| Wi-Fi | 20–100 m | High | High | Poor as the main sensor link because coverage and power needs become difficult over 2 km. |
| Bluetooth | 8–10 m | Low to medium | Moderate | Poor for a factory-wide layout because the range is too short. |
| LoRa | Up to long-range, far beyond factory-scale areas | Very low | Low | Strong fit for distributed sensors sending small periodic readings. |

| | | | | |
|---|---|---|---|---|
| MQTT | Not a radio link; it is a messaging protocol | Very light overhead | Depends on network | Excellent between gateway and app/cloud, but it needs an underlying network such as Wi-Fi, Ethernet, or cellular. |

A practical design is: **sensors → LoRa → gateway → MQTT → server/app**. The LoRa links handle the difficult 2 km distributed sensing problem, and the gateway can publish the collected data through MQTT to the factory dashboard or cloud platform

Wi-Fi is attractive for speed, but the power consumption and coverage requirements make it less suitable for 500 widely distributed sensors. Bluetooth is even less suitable because its short range limits it to local point-to-point or room-level use.MQTT should not be treated as a replacement for Wi-Fi or LoRa; it sits above the network layer and solves a different problem, namely efficient message delivery from devices to applications. That is why the combination of **LoRa + MQTT** is usually the most sensible architecture here.

Use **LoRa for sensor communication** and **MQTT for data transport from gateway to software systems**. If some sensors are very close to access points and need high throughput, you could use Wi-Fi for those few nodes, but not for the whole 2 km, 500-sensor deployment.

Q3) A temperature sensor suddenly starts showing impossible values (e.g., –120°C or 300°C). How you would determine whether the problem lies in the sensor, wiring, power supply, ADC, or software.

**1. Check the Software**

Software issues are the easiest to verify first without modifying the physical hardware circuit.

- **Action:** Print the raw digital or analog readings (ADC counts or raw hex codes) to the serial monitor before any mathematical conversion equations are applied.
- **Diagnosis:** * If the raw readings look completely stable and normal, but the output temperature reads exactly −120◦C or 300◦C, the problem lies in a **software bug** (e.g., a divide-by-zero error, an integer overflow).

If the raw values themselves are maxed out, floating, or zeroed out, the issue is hardware-related.

**2. Inspect the Wiring & Connections**

Loose connections or broken wires often trigger extreme sensor outputs due to open or short circuits.

- **Action:** Visually inspect all connections and use a multimeter to run a **continuity test** across the data, VCC, and GND lines.
- **Diagnosis:** * **Open Circuit (Broken Wire/Loose Pin):** If the signal wire or pull-up/pull-down resistor is disconnected, the microcontroller pin floats. For many digital protocols (like 1-Wire used by the DS18B20), a disconnected line defaults the software reading to an extreme error code like −127∘C or 85∘C.
    - **Short Circuit:** If the data line is directly shorted to GND or VCC, the reading will stay permanently locked at its absolute minimum or maximum value (mimicking −120∘C or 300∘C).

**3. Verify the Power Supply**

Electronic sensors rely on stable reference voltages to provide accurate readings.

- **Action:** Measure the voltage directly across the sensor's VCC and GND pins using a multimeter while the system is fully running.
- **Diagnosis:** * If the voltage drops below the sensor's minimum operating threshold (e.g., drops to 2.5V for a 3.3V sensor), the internal logic of a digital sensor or the output bridge of an analog sensor fails, causing corrupt, erratic, or maxed-out data.
    - Check for high AC ripple/noise on the DC power line, which can severely disorient sensitive analog-to-digital conversions.

**4. Test the Sensor Hardware**

Isolate the sensor to confirm whether the transducer element itself has burned out or degraded.

- **Action:** Disconnect the suspect sensor from the circuit and replace it with a **known working sensor** of the exact same model, or test the suspect sensor on an entirely different, independent development board.
- **Diagnosis:**
    - If the replacement sensor immediately starts outputting accurate room-temperature readings, the original **sensor is dead or physically damaged** (e.g., due to moisture ingress, thermal stress, or internal component degradation).
    - If the replacement sensor *still* yields impossible values, the fault lies deeper in the processing system (ADC or controller).

**5. Evaluate the Analog-to-Digital Converter (ADC)**

If you are using an analog sensor (like an LM35 or a thermistor), the ADC must convert the analog voltage to a digital signal.

- **Action:** Disconnect the sensor's signal wire from the ADC input pin. Use a reliable external power source or a voltage divider to apply a known, steady DC voltage (e.g., exactly 1.5V) directly to that ADC pin.
- **Diagnosis:**
    - If the code reads the expected ADC value corresponding to 1.5V perfectly, the ADC is fully functional.
    - If the code still returns a saturated maximum value (e.g., 4095 on a 12-bit ADC) or a flat zero regardless of the input voltage, the **ADC channel is blown or misconfigured** in the hardware registers.

Q4) An ESP32-based weather station must operate on battery power for six months without maintenance. What hardware and software modifications would you make to maximize battery life?

To achieve a six-month operational lifespan on battery power, an ESP32-based weather station requires a combination of aggressive software sleep cycles and the elimination of hardware power leaks. By default, standard ESP32 development boards consume around 75 mA to 240 mA when active, which would drain a standard 18650 Li-ion battery in less than two days.

Here are the essential hardware and software modifications needed to maximize battery life:

**1. Software Modifications**

The primary software strategy is to keep the ESP32 in a low-power state for the absolute maximum percentage of time. Weather patterns change slowly, meaning the station only needs to wake up briefly (e.g., for 10 seconds every 15 minutes) to sample sensors and upload data.

- **Implement Deep Sleep Mode:** Utilize the ESP32's **Deep Sleep** state between readings. In Deep Sleep, the main CPU, Wi-Fi, and Bluetooth modules are powered down, leaving only the Ultra-Low-Power (ULP) co-processor or the Real-Time Clock (RTC) timer active. This drops the chip's current consumption from ~75 mA down to **around 10 to 15 µA**.

**Code Example:** esp_sleep_enable_timer_wakeup(TIME_TO_SLEEP * 1000000); esp_deep_sleep_start();

- **Explicitly Disable Radio Modules:** Wi-Fi and Bluetooth turn on by default during boot. Ensure they are explicitly turned off when not broadcasting data to prevent power spikes.
    - Use WiFi.mode(WIFI_OFF); and btStop(); before going to sleep.

- **Minimize Awake Time (Fast Connection Protocol):** * **Static IP:** Assign a static IP address to the ESP32 in the code. This bypasses the DHCP negotiation process connecting to Wi-Fi, saving 2 to 4 seconds of peak power consumption (~120 mA active radio current) during every wake cycle.
    - **Lightweight Data Protocols:** Use **MQTT** or UDP instead of heavy HTTPS requests to compress the data payload and minimize the time the Wi-Fi radio stays active.
- **Reduce CPU Clock Speed:** The ESP32 defaults to a clock speed of 240 MHz. For reading basic environmental sensors (like temperature, humidity, and pressure), scale the CPU frequency down to **80 MHz** using setCpuFrequencyMhz(80);. This significantly lowers active power consumption without affecting sensor acquisition.
- **Store Variables in RTC Memory:** Instead of writing to flash memory (which is power-intensive), use the RTC_DATA_ATTR attribute to save state variables or accumulate sensor readings locally across multiple sleep cycles before turning on the Wi-Fi to upload in batches.

**2. Hardware Modifications**

Software modifications are useless if the physical development board contains components that constantly leak current.

- **Use a Bare Module or Low-Power Board:** Standard development boards (like the NodeMCU ESP32) feature an onboard USB-to-UART serial converter chip (e.g., CP2102 or CH340) and an inefficient Linear Voltage Regulator (LDO like the AMS1117). These components continuously draw **10 mA to 20 mA** even during deep sleep.
    - **Solution:** Use a bare module like the **ESP32-WROOM-32** on a custom PCB, or use specialized low-power development boards (such as the FireBeetle ESP32, which is engineered to consume only 10 µA in deep sleep).
- **Sensor Power Gating (MOSFET Switching):** Many weather sensors (like the DHT22 or certain barometric sensors) draw parasitic current even when they are idle.
    - **Solution:** Do not power sensors directly from the 3.3V rail. Instead, pipe their power through a **P-channel MOSFET** or a digital GPIO pin (if the sensor draws less than 40 mA). The ESP32 can pull the pin HIGH to power the sensor when it wakes up, and pull it LOW to completely cut off the sensor's ground/power path before entering deep sleep.

- **Optimize the Voltage Regulator:** If you must use a battery with a fluctuating voltage (like a 3.7V Li-ion battery that ranges from 4.2V fully charged to 3.0V empty), swap out standard linear regulators for a **High-Efficiency Buck-Boost Switching Regulator** with a very low quiescent current (e.g., MCP1700 or HT7333-A).
- **Direct Battery Pairing (LiFePO4 Option):** Alternatively, use a **LiFePO4 battery**. These cells feature a highly stable nominal voltage of 3.2V to 3.3V. You can connect a LiFePO4 battery directly to the 3.3V and GND pins of a bare ESP32 chip, eliminating the need for a voltage regulator entirely and achieving close to 100% power efficiency.

Q5) An automatic braking system in a vehicle requires communication with almost zero delay. Which communication method would you choose and why?

To measure the average temperature of a square-base cuboid classroom (a×a×b), the optimal placement for a single sensor is at the **geometric center**: (a/2,a/2,b/2). At this central point, the sensor is maximally isolated from localized thermal variations, ensuring a representative average reading of the entire volume.

**Impact of Poor Placement:**

- **Near Windows/Doors:** Drafts or direct sunlight introduce sudden external ambient spikes.
- **Near Ceiling/Floor:** Heat naturally rises (convection), so a high sensor reads too hot, while a low one reads too cold.
- **Near AC/Heaters:** Skews data by registering immediate local output rather than ambient room temperature.

Q6) You need to measure the average temperature of a classroom using only one sensor. Where would you place the sensor and why? Explain how poor placement could affect the accuracy of your measurements. Assume the classroom to be a square-base cuboid of side length “a” and height “b”.

**1. Optimal Sensor Placement**

The sensor should be placed at the **geometric center** of the classroom. Assuming one corner is the origin (0,0,0).

**Reason:** This point is maximally isolated from external boundary walls, the floor, and the ceiling, providing the most unbiased, representative reading of the bulk air volume.

**2. Impact of Poor Placement**

Placing the sensor incorrectly introduces localized microclimate biases:

- **Too High (Ceiling):** Reads **artificially high** because warm air rises due to natural convection (thermal stratification).
- **Too Low (Floor):** Reads **artificially low** as denser, cooler air settles at the bottom.
- **Near Windows/Outer Walls:** Causes **volatile fluctuations** from direct solar radiation (radiative heating) or drafts.
- **Near HVAC/Appliances:** Registers immediate **source outputs** (hot/cold drafts or equipment heat) rather than the ambient room average.

Q7) Under what situations could an analog sensor perform better than a digital one?

**Key Scenarios Where Analog Sensors Outperform Digital Ones**

- **Zero-Latency (Real-Time Applications):** Analog sensors change voltage or current at the exact speed of physics. They lack the built-in communication overhead (like I2C or SPI) and processing delays typical of digital chips, making them perfect for high-speed systems (e.g., rapid braking or vibration monitoring).
- **Infinite Resolution:** Digital sensors are restricted by their internal Analog-to-Digital Converter (ADC) bit-depth (e.g., 8-bit or 12-bit) which creates discrete value steps. Analog sensors deliver a truly continuous signal, avoiding quantization errors.
- **Extreme Environments:** Digital sensors feature sensitive onboard silicon logic integrated circuits (ICs) that fail under intense heat, extreme moisture, or high radiation. Heavy-duty analog alternatives (like thermocouples) contain no internal microchips, making them far more rugged.
- **Ultra-Low Power or Passive Operation:** Many analog components (like basic thermistors or LDRs) are passive. They draw negligible power and can easily be completely power-gated, ideal for sparse energy-harvesting nodes.
- **Processor-Free Hardware Loops:** Analog sensors can directly drive hardware components (like routing a voltage output directly into a transistor or analog comparator to trigger a cooling fan). Digital sensors require a microcontroller to parse and decode their data stream.

Q8) Draw the complete architecture/block diagram of your IoT project, beginning from the sensor and ending at the user interface. Identify every possible point where a failure could occur and suggest one solution for each.

Q9) If an attacker gains access to your IoT device through the Wi-Fi network, what kinds of attacks could they perform? Suggest at least five methods to improve the security of the system (with applicable solutions, do not cite far-fetched ideas)

**1. Potential Attacks via Wi-Fi Access**

If an attacker breaches the network, they could execute:

- **Device Hijacking:** Taking unauthorized control of physical actuators (e.g., unlocking doors or turning off pumps).
- **Data Interception (Eavesdropping):** Stealing unencrypted, sensitive telemetry data or passwords.
- **Lateral Movement:** Using the compromised IoT device as a stepping stone to attack secure laptops or servers on the same local network.
- **Botnet Recruitment:** Infecting the device with malware to participate in Distributed Denial of Service (DDoS) attacks.
- **Firmware Tampering:** Uploading malicious code or erasing flash memory to permanently "brick" the device.

**2. Five Methods to Improve Security**

1. **Implement Encryption (TLS/SSL):** Use secure protocols like **HTTPS** or **MQTTS** to encrypt all data in transit, rendering intercepted data unreadable.
2. **Eliminate Default Passwords:** Never hardcode default credentials. Force the user to set a strong, unique password during the initial device setup.
3. **Require Signed OTA Updates:** Ensure all Over-The-Air firmware updates are cryptographically signed so the device rejects any unauthorized or modified code.
4. **Network Segmentation:** Connect IoT devices to an isolated **Guest Wi-Fi Network** or VLAN to separate them from personal computers and critical data.

5. **Disable Unused Services:** Reduce the attack surface by closing unnecessary network ports and disabling unused protocols (like Telnet or open HTTP servers) before deployment.

Q10) Your ESP32 must simultaneously interface with a DHT11 sensor, an ultrasonic sensor, an OLED display, a relay module, and a soil moisture sensor. What hardware or software conflicts could occur, and how would you solve them?

**1. Hardware Conflicts and Their Solutions**

**Boot Issues (Strapping Pins):**
Some GPIO pins on the ESP32, such as **GPIO 0, 2, and 12**, are used during the boot process. If devices like relay modules are connected to these pins, they can interfere with booting and prevent the ESP32 from starting properly. To avoid this, connect peripherals to safe GPIO pins such as **GPIO 4, 13, 14, 25, or 26**.

**ADC2 and Wi-Fi Conflict:**
If the soil moisture sensor is connected to an **ADC2** pin, it may stop working whenever Wi-Fi is enabled because ADC2 is shared with the Wi-Fi hardware. A better solution is to connect the sensor to an **ADC1** pin (GPIO 32–39), which works normally even when Wi-Fi is active.

**Voltage Drops (Brownouts):**
Devices like relay modules and OLED displays can draw a sudden burst of current. If they are powered directly from the ESP32's 3.3V pin, the voltage may drop, causing the board to restart repeatedly. This problem can be avoided by powering high-current devices, such as relays, from a separate **5V power supply** while sharing a common ground.

**2. Software Conflicts and Their Solutions**

**Blocking Code and Timing Issues:**
Using functions like `delay()` or frequently updating the OLED display can block the processor, causing the ESP32 to miss the precise timing required by sensors such as the **DHT11** and **ultrasonic sensor**. Instead of `delay()`, use **millis()-based non-blocking timing** and hardware interrupts (or libraries like **NewPing**) to ensure smooth and accurate sensor readings.

**Slow Performance Due to Sequential Execution:**
Running all tasks one after another in a single loop can make the system slow and unresponsive, especially when handling Wi-Fi communication and multiple sensors. Since the ESP32 has two processor cores, **FreeRTOS** can be used to distribute the workload. Time-critical tasks like sensor readings can run on one core, while Wi-Fi

communication and OLED updates can run on the other, improving overall performance and responsiveness.

Q11) An ESP32 uploads sensor data to the cloud every minute. The internet connection fails for 12 hours. Design a strategy that ensures no important data is permanently lost.

- **Strategy to Prevent Data Loss**
- If the internet connection is unavailable for 12 hours, the ESP32 should be able to store around **720 sensor readings** (one reading every minute) without losing any data. A simple and reliable approach is as follows:
- **1. Store Data Locally:**
  Whenever the Wi-Fi connection is lost, the ESP32 should save all sensor readings to its internal flash memory using **LittleFS/SPIFFS** or to an external **SD card**. This ensures that no readings are lost during the outage.
- **2. Keep Accurate Timestamps:**
  Since the ESP32 cannot synchronize its clock with an NTP server while offline, a battery-backed **RTC module** such as the **DS3231** can be used to maintain the correct date and time. This allows every stored reading to be accurately timestamped.
- **3. Upload Data After Reconnection:**
  Once the internet connection is restored, the ESP32 should automatically upload the saved readings in small batches instead of sending everything at once. After the cloud server confirms that the data has been received successfully, the locally stored records can be deleted. This prevents both data loss and duplicate uploads.
  -

Q12) Your IoT device must operate continuously for five years without human intervention. What changes would you make to both the hardware and software to improve long-term reliability?

**Hardware Modifications**

- **Power Source:** Use long-lasting **LiFePO4 batteries** paired with a solar panel for energy harvesting.
- **High-Endurance Memory:** Replace standard Flash/EEPROM with **FRAM** to prevent memory wear-out from 5 years of constant data logging.

- **Auto-Recovery:** Add an **external hardware watchdog timer** to physically cut and restore power if the microcontroller freezes completely.
- **Environmental Protection:** Use an **IP67/IP68-rated enclosure** and apply conformal coating to the PCB to prevent moisture and dust damage.

**Software Modifications**

- **Prevent Memory Leaks:** Strictly use static arrays and **zero dynamic memory allocation** (avoid malloc() or the String class) to prevent crashes over time.
- **Fail-Safe Updates:** Implement **A/B partition OTA updates** so the device can automatically roll back to a working version if a remote update fails.
- **Network Resilience:** Use an **exponential backoff algorithm** for Wi-Fi/MQTT reconnections to prevent rapid battery drain during prolonged network outages.
- **System Health:** Program **scheduled periodic reboots** (e.g., once a month) to regularly flush the RAM and reset all network stacks.

Q13) Can an IoT system be considered "smart" if it does not use Artificial Intelligence? Justify your answer with suitable examples. What differentiates “smart” from “intelligent”?

Yes, an IoT system can definitely be considered "smart" without using AI. Here is the concise breakdown:

"Smart" Systems (No AI): These rely on basic connectivity, remote control, and pre-programmed rules (If-Then logic). Example: A smart irrigation system that automatically turns on a pump strictly when soil moisture drops below a set threshold.

"Intelligent" Systems (With AI): These use machine learning to analyze data, recognize patterns, and make autonomous decisions without explicit programming. Example: An intelligent thermostat that learns your daily routine over time and automatically adjusts the temperature before you get home. The Core Difference: "Smart" devices simply execute fixed rules over a network, whereas "intelligent" devices adapt and learn from their environment.

Q14) Compare polling and interrupt-based programming in embedded systems. In which situations would interrupts significantly improve the performance of an IoT device?

**1. Polling vs. Interrupts**

- **Polling:** The microcontroller continuously checks for an event in a loop. It consumes high CPU and power, and can cause delays if the processor is busy with other tasks.
- **Interrupts:** The microcontroller sleeps or does other work. When an event happens, a hardware signal instantly pauses the main code to run a specific function (ISR). It is highly power-efficient and offers near-zero latency.

**2. Situations Where Interrupts Improve Performance**

Interrupts significantly improve IoT systems in the following scenarios:

- **Battery-Powered Devices:** Allows the device to stay in "deep sleep" (drawing almost zero power) and wake up *only* when a physical sensor is triggered (e.g., a smart doorbell).
- **Capturing Fast Signals:** Ensures microsecond-level pulses (like reading a fast-spinning motor encoder) are never missed while the CPU is busy updating a screen.
- **Asynchronous Networking:** Immediately handles incoming, unpredictable data packets (like an MQTT message) without requiring the system to constantly ask "is data here yet?".

Q15) Every engineering design involves trade-offs. Looking at your own IoT project, identify three design decisions where you sacrificed one aspect (such as cost, speed, power, accuracy, or complexity) to improve another. Explain whether you believe those trade-offs were justified.

When I was working on the smart ID card idea, I had to make choices that weren't perfect but made sense for where I was.

**Battery life vs. features**

1. I picked the ESP32-C3 Mini because it gave me Wi-Fi and Bluetooth in one chip. That meant I could do cool things like scanning devices or sending data to a server. The downside? It eats more battery than a simpler microcontroller. I knew I'd have to recharge more often, but I felt the extra features were worth it.

**Accuracy vs. affordability**

2. For motion sensing, I went with a basic IMU instead of a high-end one. The cheaper sensor doesn't give super-precise readings, but it was affordable and easy to use. Honestly, for a student project, I didn't need aerospace-level accuracy — I just needed something that worked well enough to show the concept.

**Simplicity vs. scalability**

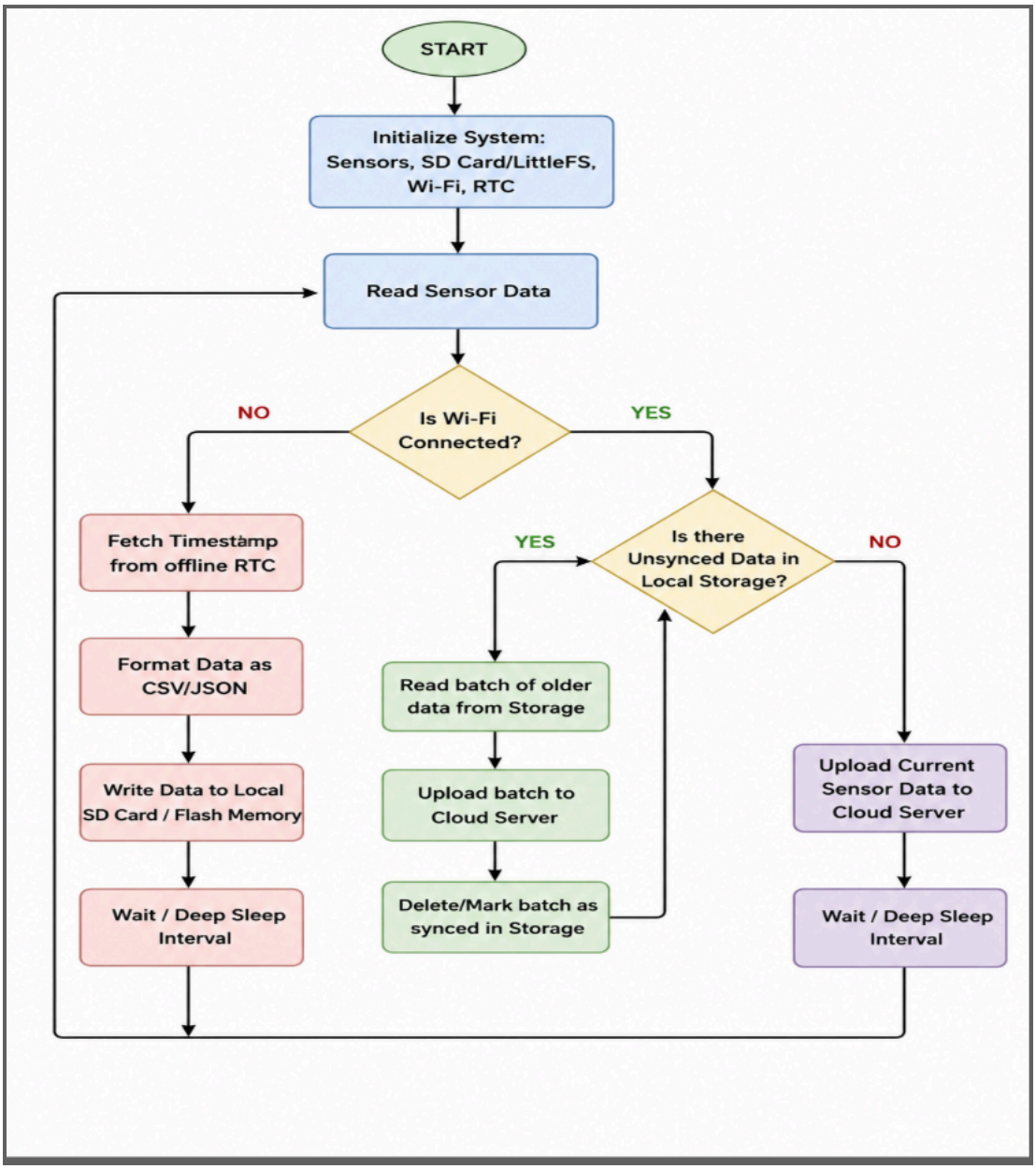
3. Instead of just sending data directly from the card to a phone, I set up MQTT. That made things more complicated to debug, but it opened the door to connecting multiple devices and building something closer to a real IoT system. It felt like a headache at times, but I knew it would pay off if I wanted to grow the project.

So yeah, each choice was a compromise. But I think they were justified because they kept the project practical, affordable, and future-ready.

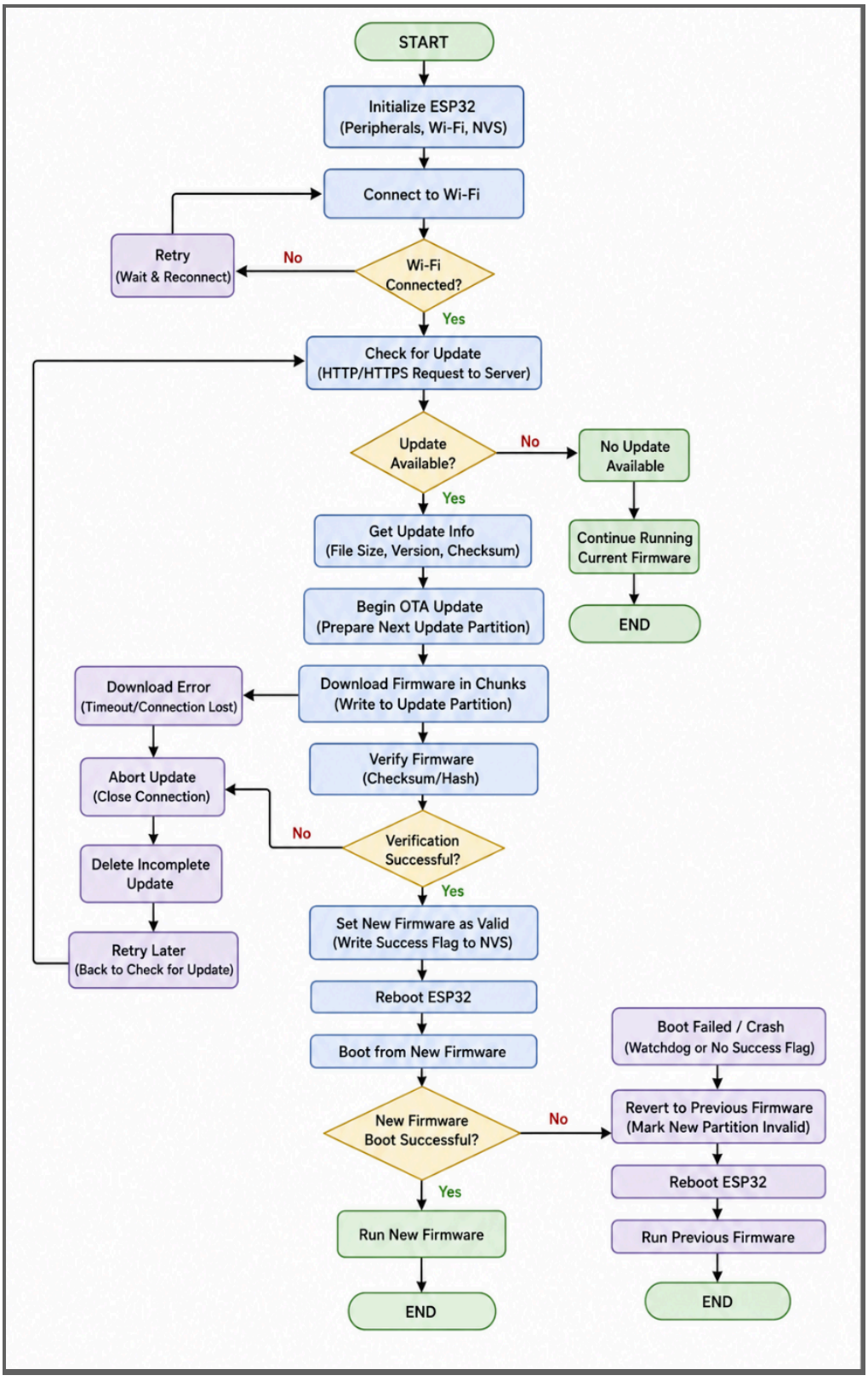
Q16) Create a flowchart for an IoT system that stores sensor readings locally whenever Wi-Fi is unavailable and automatically uploads the stored data once the connection is restored. Mention all the programs/protocols/I-O used in the system.

**Required Components**

- **Programs:** C/C++ (Arduino IDE or ESP-IDF) and **LittleFS/SPIFFS** for managing local files on the flash memory.
- **Protocols: Wi-Fi** (network connection), **MQTT** (lightweight cloud data transfer), and **SPI** (if using an external SD card).
- **I/O: ADC/GPIO** pins to read the physical sensors, and an internal **RTC (Real-Time Clock)** to timestamp the data while offline.




Q17) Draw a flowchart showing how an ESP32 performs an OTA (Over-the-Air) firmware update, including error handling if the update fails.


START
Initialize ESP32
(Peripherals, Wi-Fi, NVS)
Connect to Wi-Fi
Wi-Fi
Connected?
No
Retry
(Wait & Reconnect)
Yes
Check for Update
(HTTP/HTTPS Request to Server)
Update
Available?
No
No Update
Available
Continue Running
Current Firmware
END
Yes
Get Update Info
(File Size, Version, Checksum)
Begin OTA Update
(Prepare Next Update Partition)
Download Firmware in Chunks
(Write to Update Partition)
Download Error
(Timeout/Connection Lost)
Verify Firmware
(Checksum/Hash)
Verification
Successful?
No
Abort Update
(Close Connection)
Delete Incomplete
Update
Retry Later
(Back to Check for Update)
Yes
Set New Firmware as Valid
(Write Success Flag to NVS)
Reboot ESP32
Boot from New Firmware
New Firmware
Boot Successful?
No
Boot Failed / Crash
(Watchdog or No Success Flag)
Revert to Previous Firmware
(Mark New Partition Invalid)
Reboot ESP32
Run Previous Firmware
END
Yes
Run New Firmware
END